

Game Leaderboard & Matchmaking System

A C++ DSA-focused competitive game backend simulation that studies how data-structure choices affect lookup, matchmaking, Top-K and dynamic ranking.

unordered_map

multimap

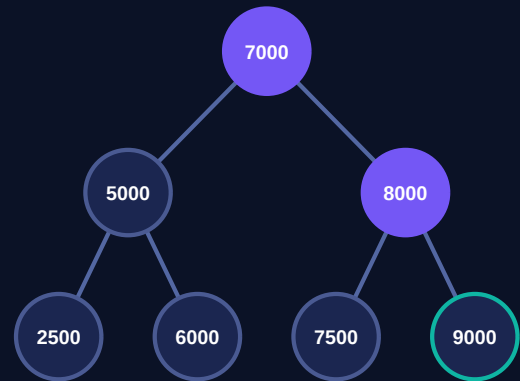
Min Heap

AVL Tree

LEADERBOARD - TOP 5

1	ProPlayer99	92%
2	xX_King_Xx	84%
3	Rahul_07	71%
4	Aman	60%
5	Karan	48%

AVL RANK TREE - $O(\log n)$



balanced - subtree sizes - rank in $O(\log n)$

$O(1)$

player lookup

$O(\log n)$

closest-rating match

$O(n \log k)$

Top-K leaderboard

$O(\log n)$

dynamic rank

~240,117x

faster rank queries at 100,000 players

AVL tree vs naive $O(n)$ scan - 1,000 queries, measured benchmark

1

Project Overview

What it is, why it exists and how every structure earns its place

A **C++ DSA-focused competitive game backend simulation** built to study how data-structure choices affect real operations: player lookup, matchmaking, Top-K retrieval and dynamic ranking. It runs as an in-memory C++ console system.

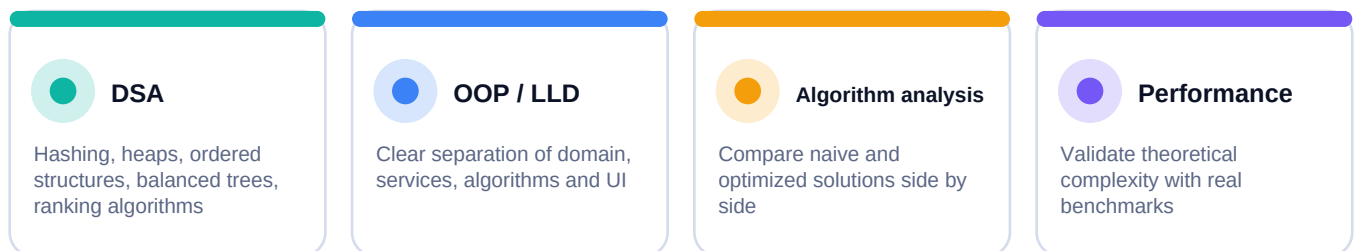
Design principle

Every major structure exists because it solves a specific problem - the goal is never to add data structures just for the sake of using them.

The engineering workflow



Project goals



What is inside this document

1	Project Overview	p. 1	10	Naive vs Optimized Ranking	p. 8
2	Core User Flow	p. 2	11	Benchmark Results	p. 8
3	Player Model	p. 2	12	Complexity Summary	p. 9
4	Win Rate & Rating	p. 3	13	Keeping Indexes in Sync	p. 9
5	System Architecture	p. 4	14	Testing & Edge Cases	p. 10
6	Data Structures	p. 5	15	Project Structure, Build & Run	p. 10
7	Closest-Rating Matchmaking	p. 6	16	Trade-offs	p. 11
8	Top-K with a Min Heap	p. 6	17	Scope, Limitations & Future	p. 12
9	AVL Tree - Dynamic Ranking	p. 7	18	Interview Talking Points	p. 12

2

Core User Flow

From registration to a refreshed leaderboard - fully automatic after one input



Fully automatic

The user does **not** manually select an opponent, winner, rating or score. Only the username is entered; the system finds the opponent, generates WIN / LOSS / DRAW, and updates every statistic.

3

Player Model

Identified only by username - derived values are calculated, never stored

class Player

username	string	unique identity
rating	int	= 400
wins	int	= 0
losses	int	= 0
draws	int	= 0
DERIVED (calculated, never stored)		
getWinRate()	double	wins / total
totalMatches()	int	W + L + D
winRateBasisPoints()	int	round(rate*10000)

Username rules

- Username is the only identity (no numeric Player ID)
- Must be unique across the system
- Empty usernames are rejected
- Spaces are not allowed
- Normal username characters allowed - validation is kept simple

Valid examples

Rahul

Rahul123

Rahul_07

ProPlayer99

xX_King_Xx

Initial state

Rating = 400

Wins = 0

Losses = 0

Draws = 0

Win Rate = 0%

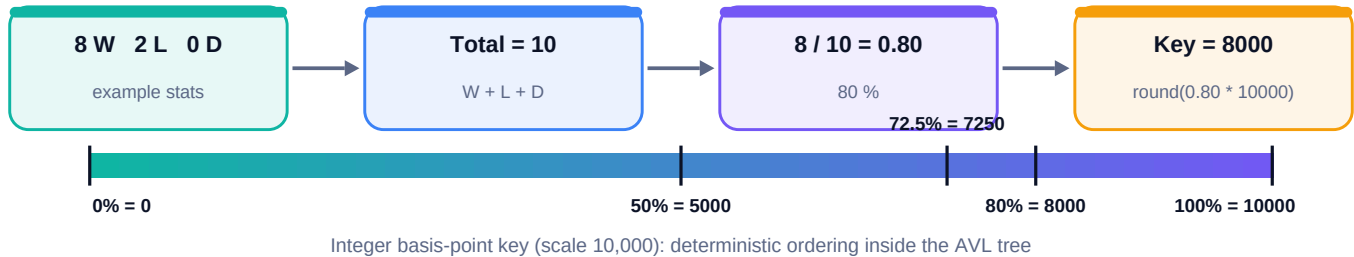
4

Win Rate & Rating

Performance versus matchmaking skill - two different concepts

$$\text{Win Rate} = \text{Wins} / (\text{Wins} + \text{Losses} + \text{Draws})$$

No matches played -> Total = 0 -> Win Rate = 0 (no division by zero, no artificial denominator)



Why Win Rate is calculated, not stored

Win Rate is derived from match statistics instead of being an independent mutable field. That prevents inconsistent states such as Wins = 8, Losses = 2, Draws = 0, Win Rate = 63%. The value is always recomputed from the actual statistics.

Metric	Purpose	Initial	Changes after a match
Rating	Matchmaking skill level	400	Win +20, Loss -20, Draw 0
Win Rate	Player performance	0%	Derived automatically from match results

Match result rules



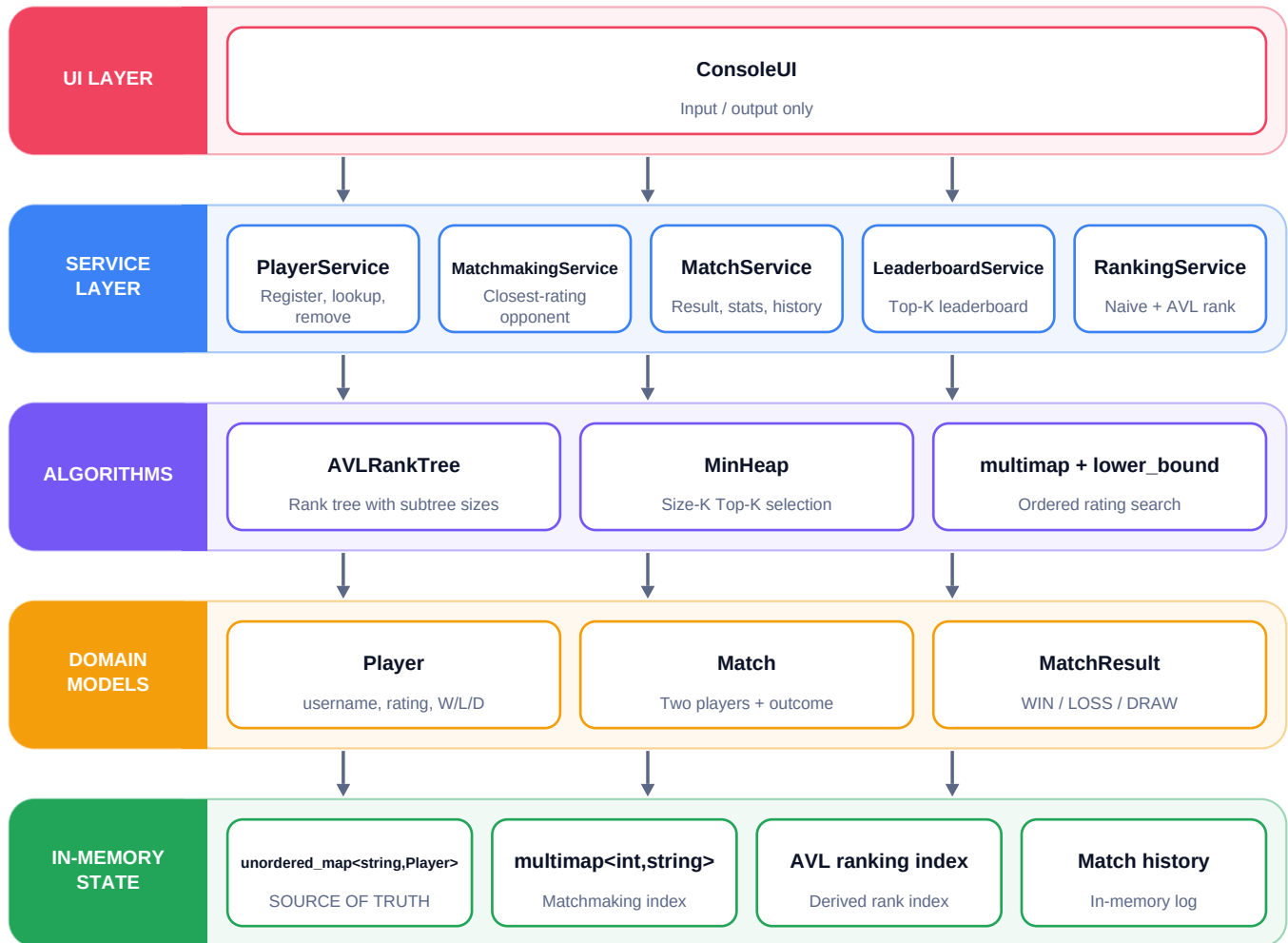
Ranking key note

Floating-point values are not used directly as the AVL ordering key. For duplicate Win Rates the username is only a deterministic internal tie-breaker - **username never affects the player's actual rank.**

5

System Architecture

Modular layers - business logic never lives in main.cpp or the console layer



calls flow downward only - business logic never lives in the console layer

Layer responsibilities

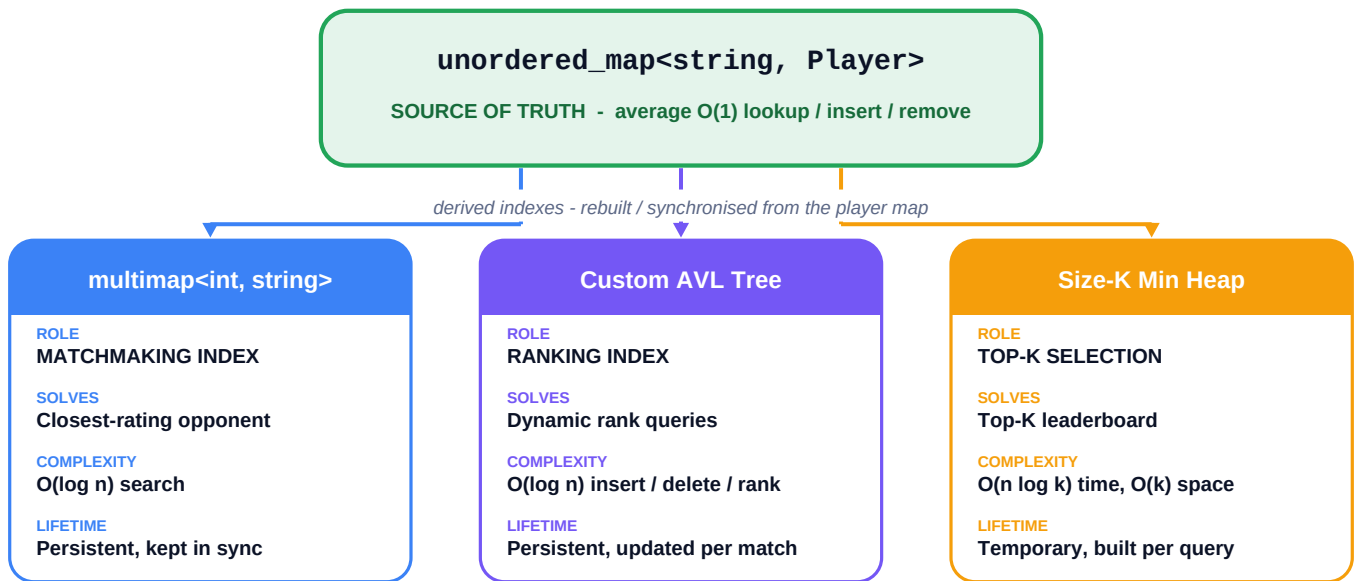
ConsoleUI	Input / output only
PlayerService	Player registration, lookup and removal
MatchmakingService	Closest-rating opponent selection
MatchService	Match lifecycle, result generation, stats + rating updates, match history
LeaderboardService	Top-K leaderboard
RankingService	Naive + AVL ranking
Player / Match	Domain models

This separation keeps business logic out of the console layer and makes the algorithm implementations independently testable.

6

Data Structures

One source of truth, three purpose-built derived structures



1 unordered_map<string, Player> - primary storage

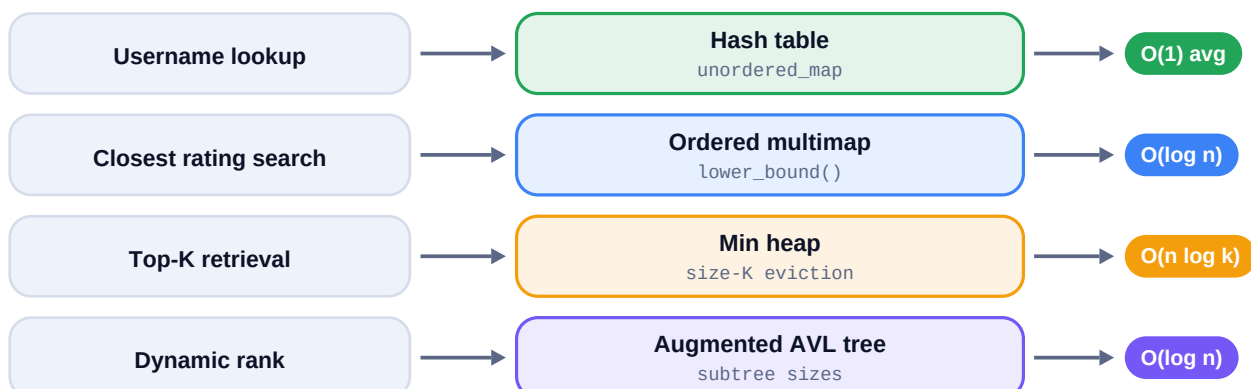
Primary player storage and fast username-based lookup. Used for Register, Find Player, View Profile, Remove Player and reading current player state. It remains the **source of truth**. Average lookup, insertion and removal are all **O(1)**.

2 multimap<int, string> - matchmaking index

Efficient closest-rating matchmaking. A multimap is used because multiple players can share the same rating, so duplicate keys must be supported. Search is **O(log n)**. It is a derived index that must stay synchronized with rating changes.

```
std::multimap<int, std::string> playersByRating;
// 390 -> Aman   400 -> Rahul   400 -> Ravi   420 -> Karan   600 -> Vikas
```

Design decisions - why not one structure for everything?



Trying to force one structure to solve all operations would make the design less suitable for the actual problems.

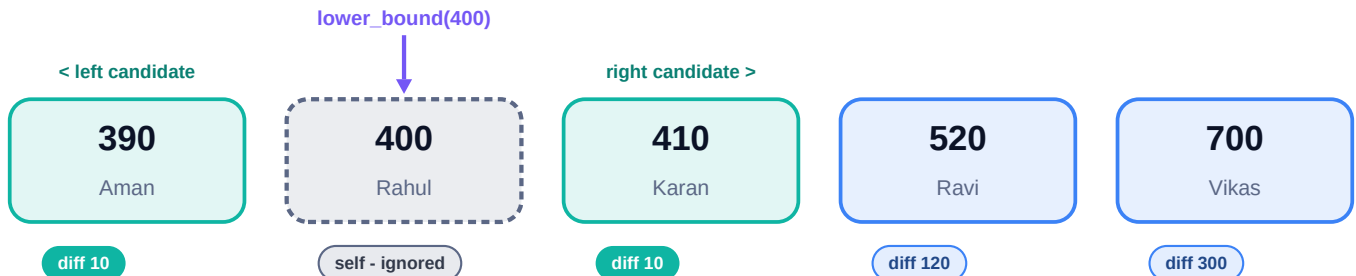
7

Closest-Rating Matchmaking

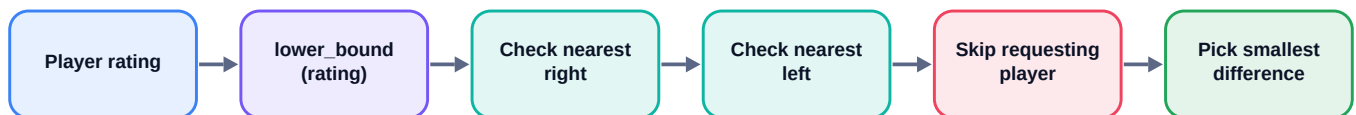
`multimap + lower_bound()` finds the nearest opponent in $O(\log n)$

The system automatically chooses an opponent whose rating is closest to the requesting player's rating, instead of scanning every player.

`multimap<int,string> playersByRating` (sorted by rating)



Requesting player: Rahul (400) -> Aman and Karan tie at distance 10 - either is a valid closest-rated opponent



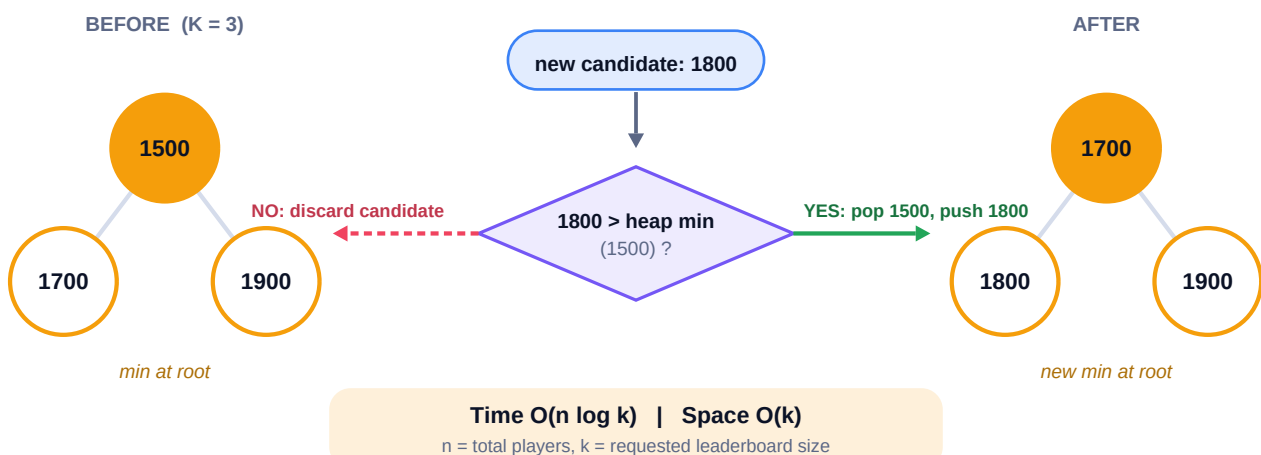
Guarantees

Self-matches are prevented, and the case where **no other player exists** is handled explicitly. Equal ratings and ties in distance are valid inputs.

8

Top-K with a Min Heap

Keep only the K best candidates - evict the smallest when a better one arrives



Why a min heap?

For the current K best players, the smallest candidate is the easiest one to remove when a better player arrives. Compared with sorting everything in $O(n \log n)$, the heap gives $O(n \log k)$ time and $O(k)$ space. The heap is temporary, derived data - the player map remains the source of truth.

9

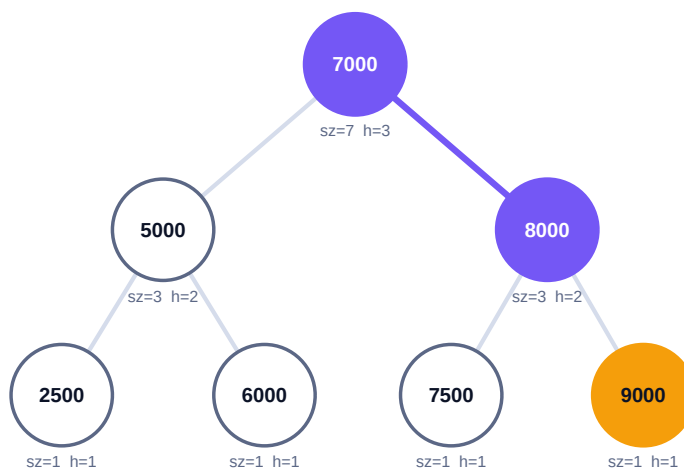
AVL Tree - Dynamic Ranking

Augmented with subtree sizes for $O(\log n)$ rank queries (V3 optimization)

Each AVL node keeps enough information for ordered ranking and subtree-size based rank queries.

Node field	Meaning
Win Rate key	Integer basis-point key used for ordering
Username	Deterministic internal tie-breaker only
Height	Used to keep the tree balanced via rotations
Subtree Size	$1 + \text{left size} + \text{right size}$ - counts players above a given player

Rank query walkthrough



rank(8000)

- 1 At 7000: $8000 > 7000$, so go right (root is not greater).
- 2 At 8000: match found. Players greater = $\text{size}(\text{right subtree}) = 1$.
- 3 $\text{rank} = 1 + 1 = 2$ - one root-to-node walk only.

$$\text{size} = 1 + \text{size}(\text{left}) + \text{size}(\text{right})$$

$$\text{rank} = 1 + \# \text{players with strictly greater Win Rate}$$

search path

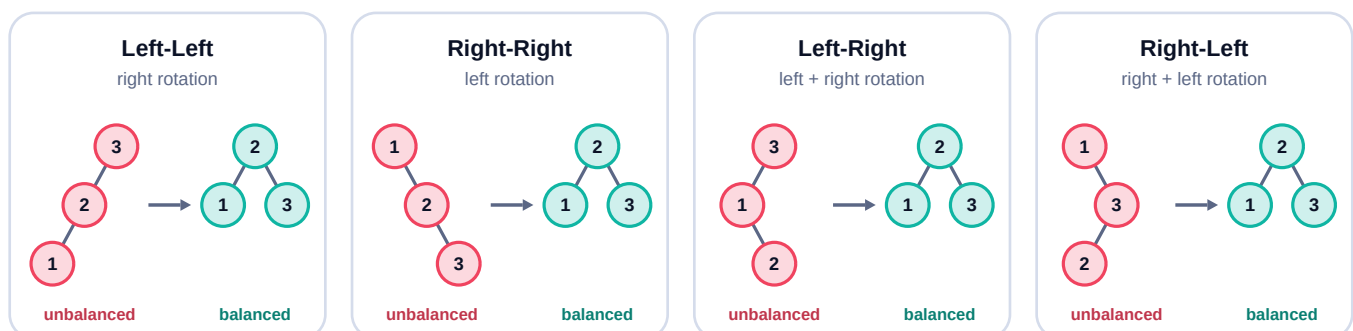
counted as greater

Competition ranking: rank = 1 + number of players with strictly greater Win Rate

Win Rate	80%	70%	70%	50%
Rank	1	2	2	4

Duplicate Win Rates receive the same rank (70%, 70% -> rank 2, 2; the next player is rank 4).

Self-balancing rotations



Insert, delete and rank are all $O(\log n)$ because rotations keep the tree height logarithmic.

10

Naive vs Optimized Ranking

Both versions are deliberately preserved

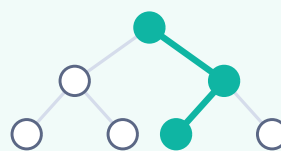
Naive ranking

 $O(n)$ 

- Scan all players
- Compare Win Rates
- Count players with greater Win Rate

kept as baseline for correctness + benchmark

Optimized ranking

 $O(\log n)$ 

- AVL tree
- + subtree sizes
- + balanced search

walks one root-to-node path

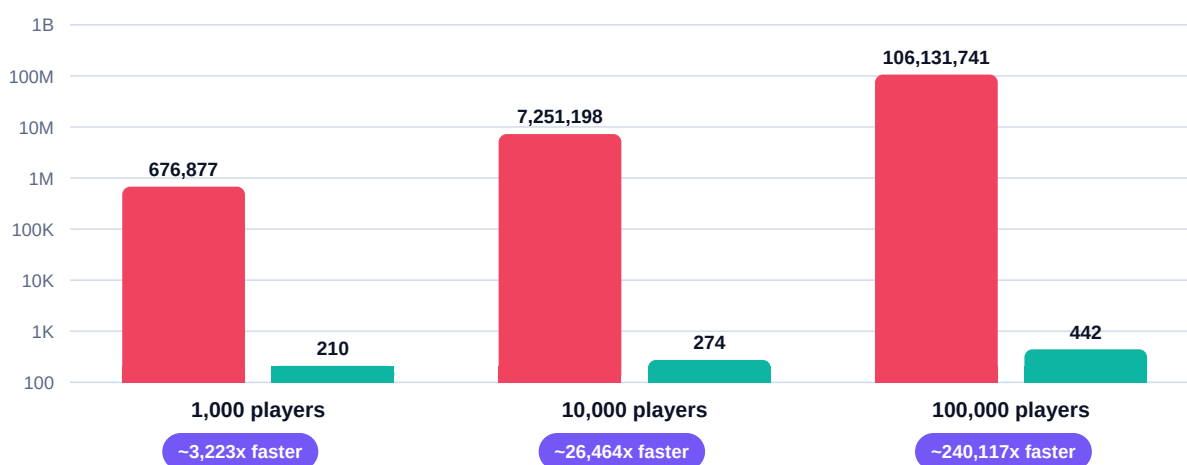
Keeping the naive implementation provides a **correctness baseline** and a fair benchmark comparison. Randomized tests assert `Naive Rank == Optimized AVL Rank` - the optimized algorithm must not merely be faster, it must return identical results.

11

Benchmark Results

1,000 rank queries at three data sizes - naive scan versus AVL tree

1,000 rank queries - total time (microseconds, log scale)

Naive $O(n)$ AVL $O(\log n)$ 

Players	Naive (microsec)	AVL (microsec)	Approx. speedup
1,000	676,877	210	~3,223x
10,000	7,251,198	274	~26,464x
100,000	106,131,741	442	~240,117x

Important note

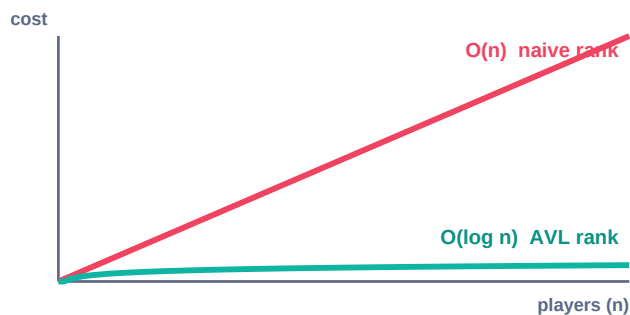
These numbers are **machine- and build-dependent**. They describe the project's current benchmark environment and are not universal hardware-independent guarantees. The theoretical comparison remains Naive $O(n)$ versus AVL $O(\log n)$.

12

Complexity Summary

n = number of players, k = requested Top-K size

Operation	Data structure / approach	Complexity
Player lookup	unordered_map	Average $O(1)$
Player insertion	unordered_map	Average $O(1)$
Player removal	unordered_map	Average $O(1)$
Closest-rating matchmaking	multimap + lower_bound()	$O(\log n)$
Top-K Win Rate	Size-K min heap	$O(n \log k)$
Top-K extra space	Min heap	$O(k)$
Naive rank	Full scan	$O(n)$
AVL insert	AVL tree	$O(\log n)$
AVL delete	AVL tree	$O(\log n)$
Optimized rank	AVL + subtree size	$O(\log n)$
Rank update after match	AVL delete + insert	$O(\log n)$



Why the gap matters

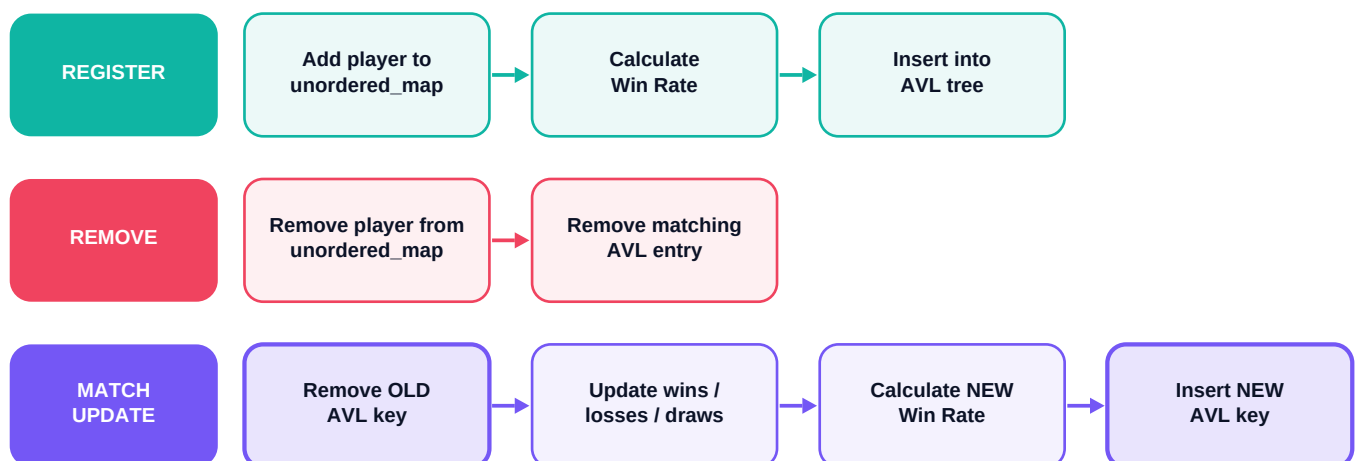
A full scan grows linearly with the player count - doubling the players doubles the cost of every rank query. A balanced tree grows only with its height, so the cost is almost flat even at hundreds of thousands of players.

That is exactly what the benchmark shows: the naive time grows ~157,000x from 1,000 to 100,000 players, while the AVL time only roughly doubles.

13

Keeping Indexes in Sync

The AVL ranking index must follow every change to player statistics



old key is removed BEFORE the new key is inserted - ordering stays valid

14

Testing & Edge Cases

Correctness tests plus DSA invariants, cross-checked against the naive baseline

Match / player behavior

- Username registration
- Duplicate usernames
- Initial rating
- Match results
- Wins / losses / draws
- Win Rate
- Closest-rated matchmaking
- Self-match prevention
- No-opponent case
- Player removal
- Match-driven updates

AVL tree

- Empty tree, single node, multiple nodes
- Increasing / decreasing insertion
- Left, right, left-right, right-left rotation
- Duplicate Win Rates
- Competition ranking
- Leaf / one-child / multi-child / root deletion
- Subtree-size validation
- Rank after insert, delete, update
- AVL invariant validation

Correctness cross-check

Randomized tests compare **Naive Rank == Optimized AVL Rank** across many scenarios.

Edge cases explicitly handled

0 matches

Duplicate usernames

No available opponent

Self-match

Same rating

Same Win Rate

Empty leaderboard

K = 0

K > number of players

Player removal

Win Rate changes after match

15

Project Structure, Build & Run

CMake project with separate include / src / tests / benchmarks

```

DSA_Projects/
|-- GameLeaderBoard/
|   |-- README.md
|   |-- CMakeLists.txt
|   |-- include/
|   |   |-- algorithms/   AVLRankTree.h, MinHeap.h
|   |   |-- models/       Player.h, Match.h, MatchResult.h
|   |   |-- services/     PlayerService.h, LeaderboardService.h, MatchService.h, RankingService.h
|   |   |-- ui/           ConsoleUI.h
|   |-- src/
|   |   |-- algorithms/   AVLRankTree.cpp
|   |   |-- models/       Player.cpp, Match.cpp, MatchResult.cpp
|   |   |-- services/     PlayerService.cpp, LeaderboardService.cpp, MatchService.cpp, RankingService.cpp
|   |   |-- ui/           ConsoleUI.cpp
|   |-- tests/            test_main.cpp
|   |-- benchmarks/      ranking_benchmark.cpp

```

Build & run

```
# configure
cmake -S . -B build

# build
cmake --build build

# run tests (Visual Studio multi-config generator)
ctest --test-dir build -C Debug --output-on-failure

# run the application
.\build\Debug\game_leaderboard.exe
```

Example console session

===== GAME LEADERBOARD SYSTEM =====

1. Register Player
2. Find Player
3. View Player Profile
4. View Leaderboard
5. Find My Rank
6. Play Match
7. View Match History
8. Remove Player
0. Exit

```
> Register Player
Enter username: Rahul
Player registered successfully.
Rating: 400
Win Rate: 0%

> Play Match
Enter username: Rahul
Finding closest-rated opponent...
Opponent found: Aman
Match result: Rahul wins
```

16

Trade-offs

Every structure buys something and costs something

unordered_map

PROS

- Very fast average lookup
- Simple username-based access

TRADE-OFF

- No ordering - cannot answer closest-rating or rank queries alone

multimap

PROS

- Keeps ratings ordered
- Efficient nearby search
- Supports duplicate ratings

TRADE-OFF

- Must stay synchronized with player rating changes

Min Heap

PROS

- Efficient Top-K selection
- Memory proportional to K

TRADE-OFF

- Not a complete sorted representation of all players

AVL Tree

PROS

- Guaranteed logarithmic height
- Dynamic insertion / deletion
- Efficient rank via subtree sizes

TRADE-OFF

- More implementation complexity
- Careful rotation + metadata maintenance
- Derived index must stay synchronized

17

Scope, Limitations & Future

Intentionally in-memory - infrastructure concerns come later

IN SCOPE (current milestone)

- C++ in-memory console system
- Source of truth: `unordered_map<string, Player>`
- Derived: multimap, AVL tree, min heap
- Match history stored in memory
- Data is lost when the app exits

INTENTIONALLY NOT INCLUDED

- PostgreSQL / Redis
- REST API / React frontend
- Microservices
- Distributed matchmaking
- Network concurrency

Possible future DSA directions

Better matchmaking structures

Advanced ranking techniques

More extensive stress benchmarks

Tournament-specific ranking

Range queries

Leaderboard history

Each should be introduced only when there is a demonstrated need: **real problem** -> **naive solution** -> **measure the bottleneck** -> **choose a DSA** -> **implement** -> **test** -> **benchmark**.

18

Interview Talking Points

Concise answers for the most likely design questions

Why <code>unordered_map</code> ?	Player lookup is username-based and needs no ordering, so a hash table gives average $O(1)$ lookup.
Why multimap for matchmaking?	Matchmaking needs the closest rating, so ordered keys allow lower_bound-based neighbour search in $O(\log n)$. A multimap also supports duplicate ratings.
Why a min heap for Top-K?	Only the K best candidates need to be retained. A size-K min heap evicts the smallest efficiently, giving $O(n \log k)$ instead of sorting everything in $O(n \log n)$.
Why AVL for ranking?	The naive rank scans every player in $O(n)$. An AVL tree with subtree sizes answers dynamic rank queries in $O(\log n)$ while staying balanced.
Why integer Win Rate keys?	Floating-point comparisons can be problematic for deterministic ordered structures, so Win Rate is converted to a fixed integer key using a 10,000 scale.
Why keep the naive version?	It provides a correctness baseline and allows a real benchmark comparison with the optimized implementation.

PROJECT PHILOSOPHY

Use the simplest correct data structure first, then optimize only when a real complexity problem exists.

- What real problem does it solve?
- What complexity improvement does it give?
- How was correctness verified?
- Why was the simpler approach insufficient?
- What trade-off does it introduce?
- What does the benchmark show?